
Table of Contents

.....	1
Problem 1	1
Problem 2.(a)	2
Problem 2.(b)	4

% Notes:

% 1. The backslash method of MATLAB uses different numerical methods to find the solution of $Ax = b$. You can type in 'help \ ' in the command window to see formatting for it. If you click on the link leading to documentation, there you will be able to see the methods used by the operator for different cases of A!

% 2. For the plot of Vandermonde Matrix -

% You will see a break at $n = 8$. If you run the code separately for that case and find out the error, you will see it is 0. That is why this is observed in the graph.

% Warnings are raised for the last few values of n as well. This is because their condition numbers are very large, and hence there is a loss in accuracy in the methods used by the backslash operator.

Problem 1

```
plot_hilb(3,11);
```

```
%-----Functions-----%
```

```
function [e,k] = Error_hilb(n)
```

```
    A = hilb(n);
```

```
    x_true = ones(n,1); % creates a column vector of ones
```

```
    b = A*x_true;
```

```
    x_comp = A\b;
```

```
    x_error = abs(x_comp - x_true);
```

```
    e = max(x_error); % infinity norm is the maximum of the row sums (which is the maximum element in a column vector)
```

```
    k = cond(A);
```

```
end
```

```
function [] = plot_hilb(n1,n2)
```

```
    l = n2 - n1 + 1; % number of elements in vectors N, E and K
```

```
    N = n1:n2;
```

```
    E = zeros(1,l); % creating the vector that will hold all errors
```

```
    K = zeros(1,l); % creating the vector that will hold all condition numbers
```

```
    % filling in the values of errors, and condition numbers -
```

```
    for i = 1:l
```

```
        [E(i), K(i)] = Error_hilb(N(i));
```

```
    end
```

```
    figure(1)
```

```
    % plotting the graphs on a loglog scale -
```

```
    subplot(1,2,1)
```

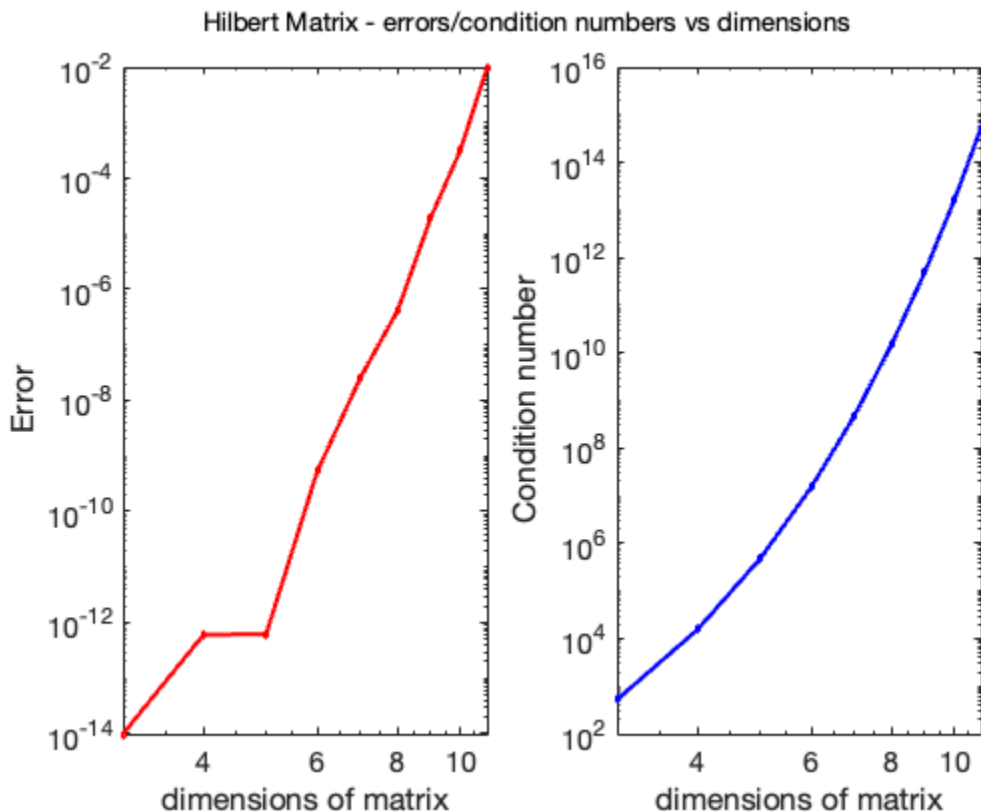
```
    loglog(N, E, 'r-o', 'LineWidth', 2, 'MarkerSize', 3);
```

```

xlabel('dimensions of matrix'); ylabel('Error')
set(gca, 'fontsize',14)

subplot(1,2,2)
loglog(N, K, 'b-o', 'LineWidth', 2, 'MarkerSize', 3);
% adding labels and legend for the graph -
xlabel('dimensions of matrix'); ylabel('Condition number')
set(gca, 'fontsize',14)
sgtitle('Hilbert Matrix - errors/condition numbers vs dimensions');
end
%-----%

```



Problem 2.(a)

```

plot_vand(4,17);
% Note that for n = 8, the error is zero, since the solution computed using
% the '\' operator is the exact solution. Hence, the corresponding data
% point is missing from the plot. This may occur due to exact cancellation
% and exact floating-point representation of intermediate quantities
% for n = 8, causing MATLAB's backslash operator to return the exact
% solution. Such behavior is an artifact of floating-point arithmetic
% and does not reflect the general conditioning of the problem. Check by
% running: [e,k] = Error_vand(8)
%-----Functions-----%
function [e,k] = Error_vand(n)
    % First define the matrix A

```

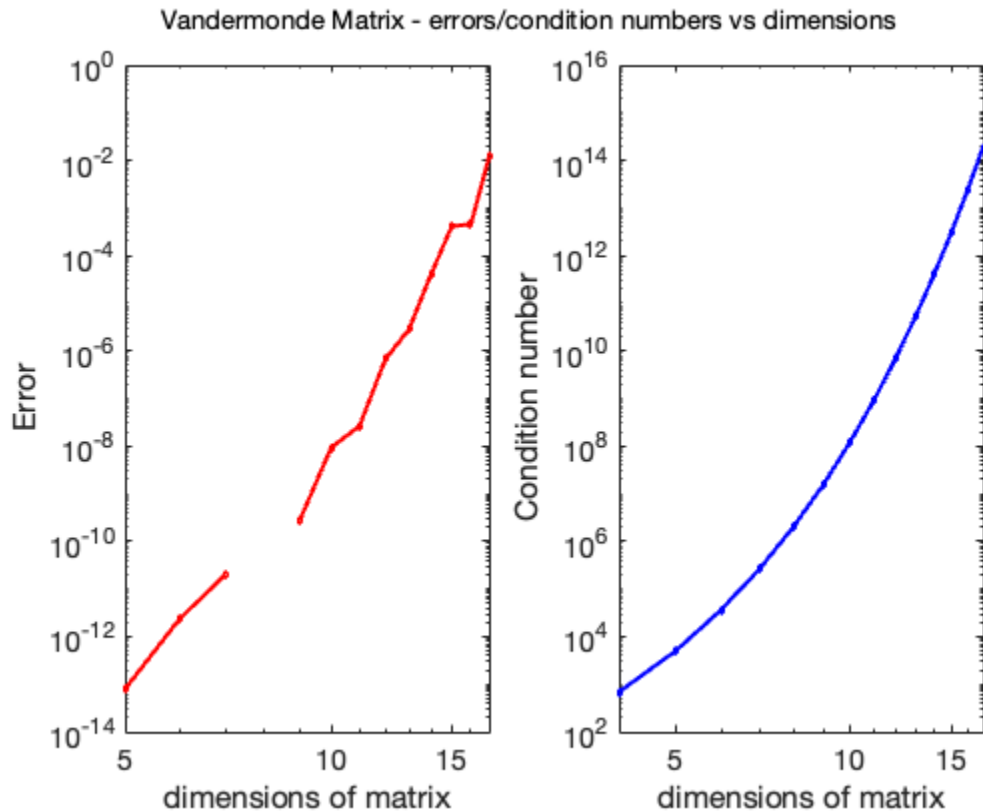
```

A = ones(n+1);
for i = 0:n
    % (i+1)th row
    x = i/n;
    for j = 2:n+1
        % jth column
        A(i+1,j) = x^(j-1);
    end
end
% Finding the condition number of A -
k = cond(A);
% Finding errors -
x_true = ones(n+1,1); % creates a column vector of ones
b = A*x_true;
x_comp = A\b;
x_error = abs(x_comp - x_true);
e = max(x_error); % infinity norm is the maximum of the row sums (which
is the maximum element in a column vector)
end

function [] = plot_vand(n1,n2)
    l = n2 - n1 + 1; % number of elements in vectors N, E and K
    N = n1:n2;
    E = zeros(1,l); % creating the vector that will hold all errors
    K = zeros(1,l); % creating the vector that will hold all condition numbers
    % filling in the values of errors, and condition numbers -
    for i = 1:l
        [E(i), K(i)] = Error_vand(N(i));
    end
    figure(2)
    % plotting the graphs on a loglog scale -
    subplot(1,2,1)
    loglog(N, E, 'r-o', 'LineWidth', 2, 'MarkerSize', 3);
    xlabel('dimensions of matrix'); ylabel('Error')
    set(gca, 'fontsize', 14)

    subplot(1,2,2)
    loglog(N, K, 'b-o', 'LineWidth', 2, 'MarkerSize', 3);
    % adding labels and legend for the graph -
    xlabel('dimensions of matrix'); ylabel('Condition number')
    set(gca, 'fontsize', 14)
    sgtitle('Vandermonde Matrix - errors/condition numbers vs dimensions');
end
%-----%

```



Problem 2.(b)

```
plot_tridiag(5,200);
%-----Functions-----%
function [e,k] = Error_tridiag(n)
    % First define the matrix A
    A = zeros(n);
    for i = 1:n
        % ith row
        A(i,i) = 2;
        if (i ~= 1)
            A(i,i-1) = -1;
        end
        if (i ~= n)
            A(i,i+1) = -1;
        end
    end
    % Finding the condition number of A
    k = cond(A);
    % Finding errors -
    x_true = ones(n,1); % creates a column vector of ones
    b = A*x_true;
    x_comp = A\b;
    x_error = abs(x_comp - x_true);
    e = max(x_error); % infinity norm is the maximum of the row sums (which
```

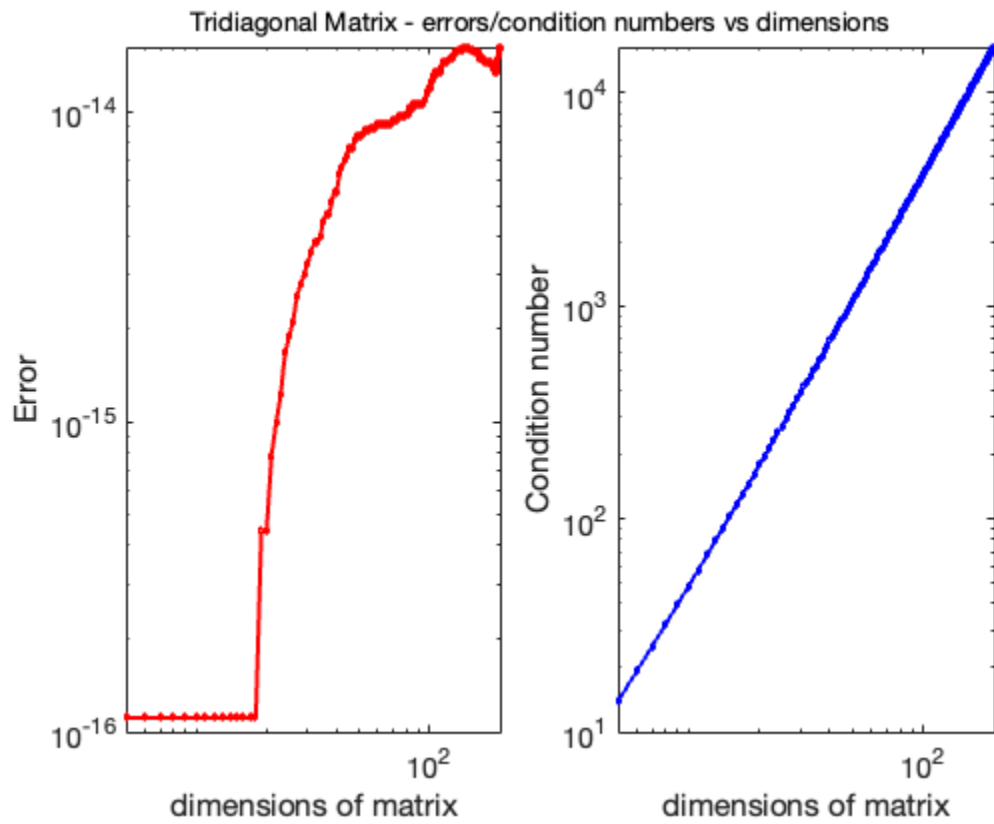
```

is the maximum element in a column vector)
end

function [] = plot_tridiag(n1,n2)
    l = n2 - n1 + 1; % number of elements in vectors N, E and K
    N = n1:n2;
    E = zeros(1,l); % creating the vector that will hold all errors
    K = zeros(1,l); % creating the vector that will hold all condition numbers
    % filling in the values of errors, and condition numbers -
    for i = 1:l
        [E(i), K(i)] = Error_tridiag(N(i));
    end
    figure(3)
    % plotting the graphs on a loglog scale -
    subplot(1,2,1)
    loglog(N, E, 'r-o', 'LineWidth', 2, 'MarkerSize', 3);
    xlabel('dimensions of matrix'); ylabel('Error')
    set(gca, 'fontsize', 14)

    subplot(1,2,2)
    loglog(N, K, 'b-o', 'LineWidth', 2, 'MarkerSize', 3);
    % adding labels and legend for the graph -
    xlabel('dimensions of matrix'); ylabel('Condition number')
    set(gca, 'fontsize', 14)
    sgtitle('Tridiagonal Matrix - errors/condition numbers vs dimensions');
end
%-----%

```



Published with MATLAB® R2024b